

Redpwn 2020 Crypto - primimity

deuterium — 2020-06-27

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/redpwn/crypto/primimity/2020-06-27-redpwn-2020-primimity.html>

Abstract

CTF writeup: Three RSA primes share a postcode in redpwnCTF 2020's primimity. Search near the modulus's cube root to find the factors.

Primity

Description

People claim that RSA with two 1024-bit primes is secure. But I trust no one. That's why I use three 1024-bit primes.

I even created my own prime generator to be extra cautious!

Files

- [primimity.py](#)
- [primimity-public-key.txt](#)

Lets take a look at source [primimity.py](#)

```
#!/usr/bin/env python3

from Crypto.Util.number import getRandomNBitInteger, isPrime

def find_next_prime(n):
    if n <= 1:
        return 2
    elif n == 2:
        return 3
    else:
        if n % 2 == 0:
            n += 1
        else:
            n += 2
        while not isPrime(n):
            n += 2
        return n

def prime_gen():
    i = getRandomNBitInteger(1024)
    d = getRandomNBitInteger(8)
    for _ in range(d):
        i = find_next_prime(i)
    p = find_next_prime(i)
    d = getRandomNBitInteger(8)
```

```

    for _ in range(d):
        i = find_next_prime(i)
    q = find_next_prime(i)
    d = getRandomNBitInteger(8)
    for _ in range(d):
        i = find_next_prime(i)
    r = find_next_prime(i)
    return (p,q,r)

def main():
    (p,q,r) = prime_gen()
    print(p)
    print(q)
    print(r)

if __name__ == '__main__':
    main()

```

As one could observe, its yet another RSA with 3 primes instead of two.

The twist with the prime generation `prime_gen` is that it doesnt generate three independent random primes, but it generates a random prime `p`, then ~~generates~~ picks two random primes `q` and `r` randomly within the **proximity** (next 256 primes max) of `p`.

How does this information help?

Cuberoor of `n` would serve as an approximation for `r`, from which we can traverse back (previous primes), till we get the prime factors of `n`

Instead of implementing `previous_prime` (lazy me), one could easily subtract some number so that the initial estimate is smaller than `p` and we could easily use `gmpy2.next_prime :p`

```

import gmpy2
n =
27396994346330977650084683711246447419234088648963962059469541961013046537721732103726
08955799251139999322976228678445908704975780068946332615022064030241384638601426716056
06712630071193343873226584683873586035325957412907461529801004732296070497215793066306
14807265669622548871449277534490425906787307790461545165496676116037927548804145266882
17305247008627664864637891883902537649625488225238118503996674292057904635593729208703
09687723127691184523383377001509321363913124438686760095611288438310543786166566627391
05667326348784646107898956072735673729337662432297986633890328071870037562261771117205
10187664096691560511459141773632683383938152396711991246874813205614169161561906148974
47851998793595031856976047424942778731086574916774091723279953809949471096483753621153
53512005203245756769870804841415613365051038728099323547485316759345274532312551323614
89570816639925234935907741385330442961877410196615649696508210921
e = 65537
c =
20829260131386741649977916055122267593628245313224330482813069835260018015819567889094
08046338065370689701410862433705395338736589120086871506362760060657440410056869674907
31420434679055461965585580566632790591276230041232337112687146304599394633192712988271
57783967649693115654071044265002848244954612525915766729896339309168370164115239834913
64869137945678029616541477271287052575817523864089061675401543733151180624855361245733
03902214032149447131893471665275816359395671191521219532867137373934212421174383585889
78952765133967833289429789037647900884950331762537778328085727173350768295399883375055
82696026111326821783912902713222712310343791755341823415393931813610365987465739339849
38017380588252202670447430854127173247803591377092218942908985292198541620284483887335
20903556850759658316634439627064737378523921078769934851639816530385885445625125974095
85410384189546449890975409183661424334789750460016306977673969147

```

```
estimate = gmpy2.iroot(n,3)[0] - 10000 # rough guess for the smallest prime
prime = gmpy2.next_prime(estimate)

for _ in range(512):
    prime = gmpy2.next_prime(prime)
    if n%prime ==0:
        print(prime)

p =
13992682289067065597719596277072694198619897349442575947682221918831637793316167375939
49018058556179399782813857089415971175310079737138467722051666592272141876229251359314
56526921198848312215276630974951050306344412865900075089120689559331322162952820292429
725303619113876104177529039691490258588465409397803
q =
13992682289067065597719596277072694198619897349442575947682221918831637793316167375939
49018058556179399782813857089415971175310079737138467722051666592272141876229251359314
56526921198848312215276630974951050306344412865900075089120689559331322162952820292429
725303619113876104177529039691490258588465409494847

r = n//(p*q)
phi = (p-1)*(q-1)*(r-1)
d = pow(e, -1, phi)
m = pow(c, d, n)
print(bytes.fromhex(hex(m)[2:]).decode())
```

flag-pr1m3`pr0x1m1ty`c4n`b3`v3ry`d4ng3r0u5"

Interactive controls are omitted from this PDF. See the canonical HTML version.